EXPERIMENT 6

Aim:
Apply K-Means and Hierarchical Clustering on sample datasets.

Theory:

1. Dataset Source

Dataset Name: Mall Customer Segmentation Dataset
Source: Kaggle
Link: https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

This dataset is widely used for applying clustering techniques to perform customer segmentation based on purchasing behavior.

2. Dataset Description

The dataset contains information about customers visiting a shopping mall. It helps in understanding customer behavior and grouping them into distinct segments based on similarities.

Dataset Size:

- Total Records: 200
- Total Features: 5

Features:

1. **CustomerID** – Unique identifier for each customer
2. **Gender** – Male or Female
3. **Age** – Age of the customer
4. **Annual Income (k$)** – Annual income in thousands of dollars
5. **Spending Score (1–100)** – Score assigned based on customer spending habits

Target Variable:
This is an **unsupervised learning dataset**, meaning there is no predefined target variable. The objective is to discover natural groupings within the data.

Key Characteristics:

- Small and well-structured dataset
- No missing or null values
- Contains both numerical and categorical features
- Ideal for clustering and segmentation tasks

3. Mathematical Formulation of Algorithms

A. K-Means Clustering

K-Means is a partition-based clustering algorithm that groups data into *k* clusters by minimizing the **Within-Cluster Sum of Squares (WCSS)**:

Where:

- *Ci*Ci = cluster i
- *μi*μi = centroid of cluster i

Algorithm Steps:

1. Randomly initialize *k* centroids
2. Assign each data point to the nearest centroid
3. Update centroids by computing the mean of assigned points
4. Repeat steps 2–3 until convergence

B. Hierarchical Clustering (Agglomerative)

Hierarchical clustering builds a tree-like structure (dendrogram) by iteratively merging clusters based on distance.

Distance Metric: Commonly uses **Euclidean distance**

Algorithm Steps:

1. Treat each data point as an individual cluster
2. Compute distances between all clusters
3. Merge the two closest clusters
4. Repeat until all points are merged into a single cluster
5. Cut the dendrogram at an appropriate level to obtain clusters

4. Algorithm Limitations

K-Means Limitations:

- Requires prior selection of the number of clusters (*k*)
- Sensitive to noise and outliers
- Assumes clusters are spherical in shape
- Results depend on initial centroid placement
- Not suitable for complex or non-linear data distributions

Hierarchical Clustering Limitations:

- Computationally expensive for large datasets
- Once clusters are merged, the process cannot be reversed
- Sensitive to noise and outliers
- Scalability issues with increasing data size

5. Methodology / Workflow

Step 1: Data Collection
Dataset was obtained from Kaggle.

Step 2: Data Preprocessing

- Removed unnecessary **CustomerID** column
- Encoded categorical variable (**Gender**)
- Selected relevant features: Age, Annual Income, Spending Score
- Applied **StandardScaler** for normalization

Step 3: K-Means Clustering

- Used **Elbow Method** to determine optimal number of clusters
- Optimal clusters identified: **k = 5**
- Applied K-Means algorithm
- Evaluated using **Silhouette Score**

Step 4: Hierarchical Clustering

- Generated **dendrogram** using Ward linkage
- Selected number of clusters: 5
- Applied Agglomerative Clustering
- Evaluated using **Silhouette Score**

Step 5: Visualization

- Visualized clusters using scatter plots
- Compared results of both clustering techniques

6. Performance Analysis

K-Means:

- Optimal clusters: 5
- Silhouette Score ≈ 0.41

Interpretation:
A silhouette score around 0.4 indicates reasonably well-defined clusters. K-Means effectively segmented customers into meaningful groups.

Hierarchical Clustering:

- Clusters formed using Ward linkage
- Produced similar grouping structure as K-Means
- Slightly lower silhouette score

Interpretation:
Although hierarchical clustering provided comparable clusters, it was slightly less efficient in terms of cluster compactness.

7. Hyperparameter Tuning

A. K-Means Parameters Tuned:

1. **n_clusters** – Tested values from 1 to 10 using Elbow Method
2. **n_init** – Set to 10 for stable centroid initialization
3. **random_state** – Fixed for reproducibility

Impact:

- Choosing k = 5 improved clustering performance
- Higher n_init ensured consistent and stable results

B. Hierarchical Clustering Parameters Tuned:

1. **n_clusters** – Determined using dendrogram
2. **linkage method** – Ward linkage selected

Impact:

- Ward method minimized intra-cluster variance
- Improved compactness and separation of clusters

Code

```
# ==============================
# CUSTOMER SEGMENTATION PROJECT
# K-Means & Hierarchical Clustering
# ==============================
# Step 1: Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
import scipy.cluster.hierarchy as sch
# Step 2: Load Dataset
df = pd.read_csv('Mall_Customers.csv')
print("First 5 rows:")
print(df.head())
```

```
print("\nDataset Info:")
print(df.info())
# Step 3: Preprocessing
# Drop CustomerID
df = df.drop('CustomerID', axis=1)
# Encode Gender
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
# Select Features for Clustering
X = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# ==============================
# K-MEANS CLUSTERING
# ==============================
# Step 4: Elbow Method
wcss = []
for i in range(1, 11):
 kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
 kmeans.fit(X_scaled)
 wcss.append(kmeans.inertia_)
plt.figure(figsize=(6,4))
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
# Choose optimal clusters (usually 5)
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
labels_kmeans = kmeans.fit_predict(X_scaled)
# Add cluster labels to dataframe
df['KMeans_Cluster'] = labels_kmeans
# Visualize KMeans Clusters
plt.figure(figsize=(6,4))
plt.scatter(X_scaled[:,1], X_scaled[:,2], c=labels_kmeans, cmap='viridis')
plt.title('K-Means Clustering')
plt.xlabel('Annual Income (scaled)')
plt.ylabel('Spending Score (scaled)')
```

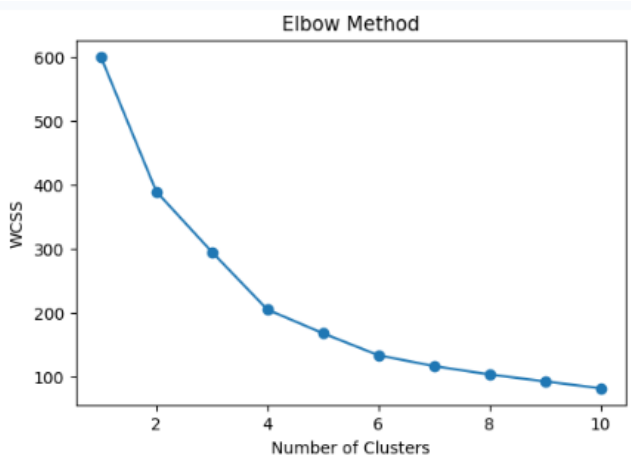
```
plt.show()
# Silhouette Score for KMeans
print("Silhouette Score (K-Means):", silhouette_score(X_scaled,
labels_kmeans))
# ==============================
# HIERARCHICAL CLUSTERING
# ==============================
# Step 5: Dendrogram
plt.figure(figsize=(8,5))
sch.dendrogram(sch.linkage(X_scaled, method='ward'))
plt.title('Dendrogram')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distance')
plt.show()
# Apply Agglomerative Clustering
hierarchical = AgglomerativeClustering(n_clusters=5, linkage='ward')
labels_hier = hierarchical.fit_predict(X_scaled)
# Add to dataframe
df['Hierarchical_Cluster'] = labels_hier
# Visualize Hierarchical Clusters
plt.figure(figsize=(6,4))
plt.scatter(X_scaled[:,1], X_scaled[:,2], c=labels_hier, cmap='rainbow')
plt.title('Hierarchical Clustering')
plt.xlabel('Annual Income (scaled)')
plt.ylabel('Spending Score (scaled)')
plt.show()
# Silhouette Score for Hierarchical
print("Silhouette Score (Hierarchical):", silhouette_score(X_scaled,
labels_hier))
# ==============================
# Final Clustered Data
# ==============================
print("\nFinal Data with Clusters:")
print(df.head())
```

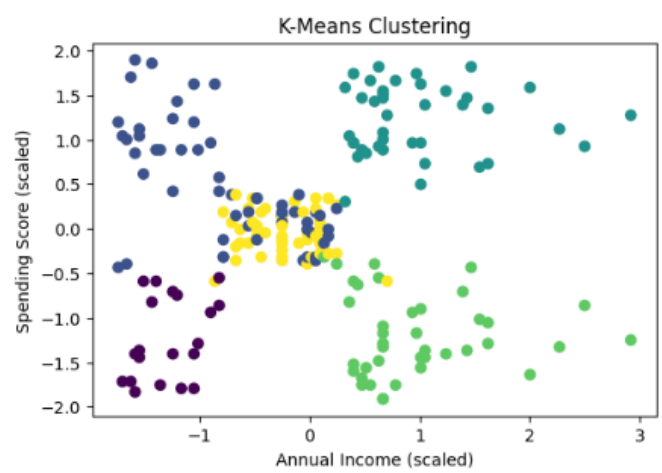
exp6 yogita.ipynb

File Edit View Insert Runtime Tools Help

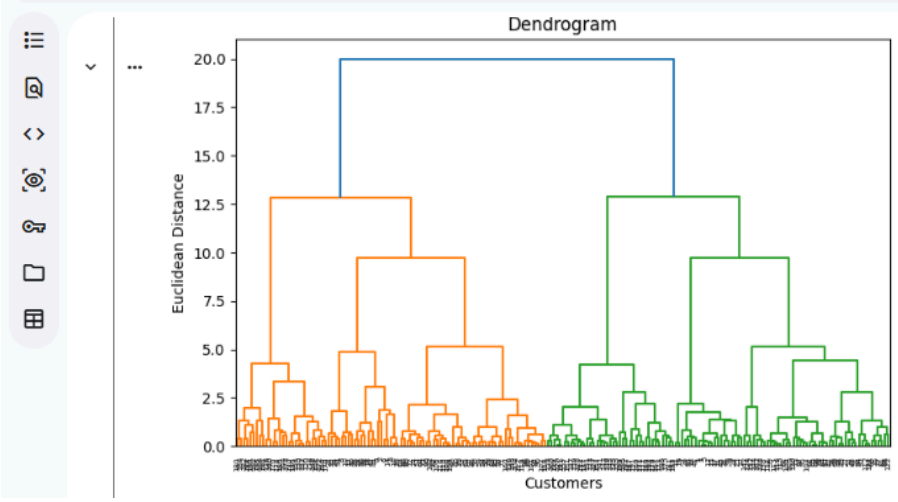
Commands + Code + Text Run all

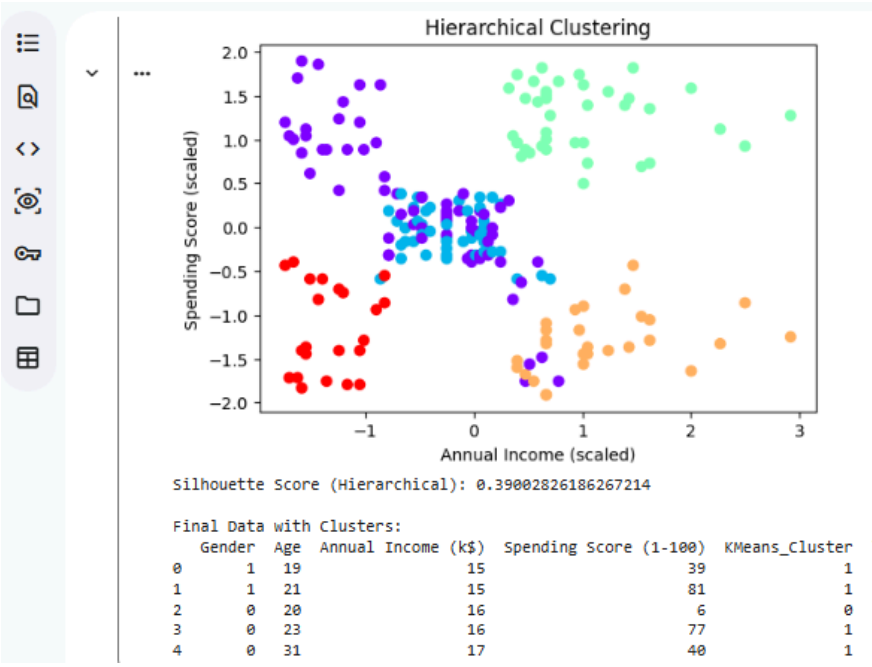
```
First 5 rows:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
None
```






```
Silhouette Score (K-Means): 0.41664341513732767
```






```
Silhouette Score (Hierarchical): 0.39002826186267214

Final Data with Clusters:
   Gender  Age  Annual Income (k$)  Spending Score (1-100)  KMeans_Cluster  \
0       1   19                  15                      39               1
1       1   21                  15                      81               1
2       0   20                  16                       6               0
3       0   23                  16                      77               1
4       0   31                  17                      40               1
```

```
   Hierarchical_Cluster
0                     4
1                     0
2                     4
3                     0
4                     4
```

Conclusion:

Both K-Means and Hierarchical Clustering effectively segmented customers into meaningful groups. K-Means performed slightly better with more compact clusters, making it more suitable for this dataset.